

A Common Interval Guided ACO Algorithm for Permutation Problems

Martin Clauß, Matthias Bernt and Martin Middendorf

Parallel Computing and Complex Systems Group

Department of Computer Science

University of Leipzig, Germany

Email: {mc, bernt, middendorf}@informatik.uni-leipzig.de

Abstract—Ant Colony Optimization (ACO) has been successfully applied to many combinatorial optimization problems. In this work we propose a new solution construction scheme for ACO. This scheme uses the common intervals of the current iteration's best solutions to guide the ants during solution construction. Firstly, we compared the performance of ACO and the proposed algorithm Common Interval ACO (CIACO). Secondly, we conducted an in-depth study for the CIACO algorithm to investigate the influence of the common interval guidance. For both experiments a large parameter space was used. The results show, that common intervals can be used to improve the solution quality in comparison to the standard ACO algorithm.

I. INTRODUCTION

Combinatorial optimization is an important field of research with many applications. A large class of problems arising in this field are permutation problems. For these problems one searches for a permutation of some element set that is minimal with respect to a given objective function. One of the best known permutation problems is the Traveling Salesman Problem (TSP). Here the element set is a set of cities with an associated distance matrix. The sought permutation of the cities specifies a cyclic tour visiting all cities once. The objective function is the sum of the distances between pairs of cities that are adjacent in the tour. Hence, an optimum solution for the TSP gives a shortest tour. Other examples for permutation problems are many scheduling problems, various vehicle routing problems, or ordering problems as, for example, the Single Row Facility Layout Problem (SRFLP) which is the second permutation problem considered in this paper. In the SRFLP an ordering of facilities is sought that minimizes the weighted distances between all pairs of facilities, where the weights reflect the amount of transmissions occurring between the facilities.

Many permutation problems, including the TSP and the SRFLP, are NP-hard [1], [2]. Therefore, the development of efficient approximation and heuristic algorithms is of importance.

Ant colony optimization (ACO) is a metaheuristic, which is inspired by the behavior of ants who use pheromones as an indirect communication channel – a phenomenon called stigmergy (see [1] and [2] for more information). Ants deposit pheromones on their trails and choose from alternative trails based on the pheromone concentration. Thereby, ants tend

to choose alternatives that have recently been preferred by other ants. This collective preference often leads to favorable solutions, e.g. short paths between nest and food source [3]. In ACO virtual ants are used to construct solutions, e.g. a tour, by depositing virtual pheromones on edges of a decision graph corresponding to the decisions made during the construction process, e.g. which city to visit next. We refer the reader to [2] for a more complete introduction to ACO.

Yet another popular heuristic are genetic algorithms (GA), see [4]. These algorithms consider a population of solutions or representations of solutions, e.g. as binary strings. The idea of genetic algorithms is to attempt to improve the solution quality iteratively by applying certain operations to the representation of the solutions. These operations are inspired by the way evolution modifies genetic information, i.e. by small modifications of single solutions (mutations) or by exchanging solution parts between solutions (crossover). This approach is straightforward if every possible representation corresponds to a solution. In the case of permutation problems, however, this is often not the case when the permutations itself are members of the population. In that case, a crossover operation which exchanges substrings of two permutations often does not yield a permutation. Therefore, more elaborate crossover operators have been devised. For the TSP, examples of such operators can be found in [5], [6]. The idea of these approaches is that subtours of different permutations can only be exchanged by a crossover operator if they contain exactly the same cities and therefore generate only valid tours. For an efficient GA that uses such a common subtour exchange operator a fast computation of the subtours is essential. The applicability of this strategy to other permutation problems was shown, for example, for the Job Shop Scheduling Problem [7].

The problem to compute common subtours efficiently has motivated the study of the following algorithmic problem. Given a set of permutations one asks for all subsets of the element set which are consecutive (i.e. they form an interval) in all of the permutations. Such subsets are called common intervals. An algorithm that computes all common intervals for two given permutations in time $\mathcal{O}(n^2)$, where n is the length of the permutations, was presented in [8]. This result was later generalized to sets of permutations and $\mathcal{O}(kn + n^2)$ time algorithms, where k is the number of permutations, have

been developed ([9], [10]). Both of these algorithms were motivated by applications in comparative genomics where the permutations are used to model the order of genes along the chromosomes and the common intervals model gene clusters. In this field the notion of common intervals has generated much attention [11], [12].

In this paper we propose a strategy for solving permutation problems that is guided by common intervals. This strategy can be used by metaheuristics that satisfy the following two properties: i) a set of good solutions is maintained and ii) new solutions are formed by sequentially adding elements to the solution. Common intervals of the best known solutions can then be used to guide the process of creating new solutions. To exemplify this common interval guidance strategy we have developed a modified ACO algorithm that is called Common Interval ACO (CIACO). The TSP and SRFLP are used as test permutation problems to investigate the proposed guidance scheme. The usefulness of the common interval guidance strategy in CIACO is evaluated empirically using real world instances of the Traveling Salesman Problem and the Single Row Facility Layout Problem.

The paper is structured as follows. Section II gives a detailed introduction of the ant colony optimization framework and its application to the considered test problems. Furthermore, common intervals are formally introduced. The CIACO algorithm is presented in Section III. The experimental setup is described in Section IV. Experimental results are presented in Section V. A conclusion is given in Section VI.

II. BACKGROUND

This section introduces ACO and its application to the TSP and the SRFLP.

A problem instance of the TSP is defined by a set of cities $N = \{1, 2, \dots, n\}$ and a distance matrix $D = [d_{ij}]_{n \times n}$, with $d_{ij} \geq 0$, that represent the pairwise distances between the cities. Solving the TSP optimally is to find a permutation π out of all permutations over N , denoted as Π_n , with:

$$\min_{\pi \in \Pi_n} \left(\sum_{i=1}^{n-1} d_{\pi(i)\pi(i+1)} \right) + d_{\pi(n)\pi(1)} \quad (1)$$

where $\pi(i)$ is the city at the i -th position of π and $d_{\pi(i)\pi(j)}$ is the distance between cities $\pi(i)$ and $\pi(j)$.

In every iteration of ACO (see Algorithm 1) each of m artificial ants walk from city to city in a randomized fashion to construct a solution of the TSP. In order to find a good solution a so called pheromone information is used. This information is stored in a pheromone matrix $[\tau_{ij}]_{n \times n}$, with $\tau_{ij} \geq 0$. The pheromone value τ_{ij} reflects the desirability to have city j after city i in a tour. The values of the pheromone matrix have been deposited by ants from previous generations that generated good solutions. In addition to the pheromone values problem-specific heuristic values $\eta_{ij} \geq 0$ are employed.

Every ant constructs a solution by starting at a random city and incrementally adds one city after another. In every construction step the next city is chosen probabilistically from

Algorithm 1 ACO algorithm for TSP (cf. [13])

- 1: Set parameters, initialize pheromone matrix
 - 2: **while** termination condition not met **do**
 - 3: Construct Solutions // for each ant
 - 4: Apply Local Search // optional
 - 5: Update Pheromone Matrix
 - 6: **end while**
-

the set S of the cities that have not been visited so far. Thus, the ant visits each city only once. The probability p_{ij} to go from city i to j follows a probability distribution called random-proportional transition rule:

$$p_{ij} = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{h \in S} \tau_{ih}^\alpha \eta_{ih}^\beta} \quad (2)$$

The parameters $\alpha, \beta \geq 0$ control the influence of the pheromone and heuristic information, respectively. In case of the TSP a commonly used heuristic is $\eta_{ij} = 1/d_{ij}$ which prefers small distances to the next city. After all ants constructed their solutions the iteration-best ant is allowed to update the pheromone matrix $[\tau_{ij}]$ by increasing the value of the τ_{ij} -entries for all pairs of cities (i, j) that are adjacent in its solution. The amount of pheromone that the iteration-best ant updates is $1/l^{ibest}$ where l^{ibest} is the length of the tour that the iteration-best ant has found. To avoid premature convergence of the algorithm, evaporation is applied before each pheromone update. This means that all pheromone values are lowered by a certain fraction $\rho \in (0, 1]$, i.e. $\tau_{ij} := (1 - \rho)\tau_{ij}$. The ACO algorithm continues until a termination criterion is satisfied. The termination criterion is, for example, a specific maximum number of iterations or the convergence of the algorithm.

In global optimization there is often a trade-off between the *exploration* of the solution space and the *exploitation* of good solutions. On the one hand, since the solution space of the TSP contains $(n - 1)!/2$ possible tours the ants should try to cover as many tours as possible. On the other hand, once a good tour is found it is often the case that concentrating on this tour by changing only small parts of it results in high quality solutions. This problem was addressed in [14] where the authors proposed a new parameter q_0 that allows to control the aforementioned trade-off. In detail, the next city s can be determined as follows:

$$s = \begin{cases} \arg \max_{h \in S} \tau_{ih}^\alpha \eta_{ih}^\beta & , \text{ if } q \leq q_0 \\ s^* & , \text{ otherwise} \end{cases} \quad (3)$$

where s^* is obtained using Equation (2), q_0 is set to a value from $[0, 1]$ and q is drawn randomly from $U(0, 1)$. Hence, low q_0 values allow the ants to explore the solution space whereas high q_0 values let the ants exploit good solutions.

The second combinatorial optimization problem we consider is the Single Row Facility Layout Problem that was first introduced by Simmons [15]. The SRFLP is the problem of

arranging rectangular facilities in a single row while minimizing the weighted distances between all facilities (for a survey on the SRFLP see [16]). Specifically, given are: n rectangular facilities $N = \{1, \dots, n\}$ each with a predefined length $l_i > 0$, $i \in N$ and communication or flow weights c_{ij} between facility i and j . The facilities must be arranged back to back on a line resulting in $n!$ possible solutions. Valid solutions for the SRFLP are therefore all permutations over N denoted as Π_n . Solving the SRFLP optimally means to find a permutation π that minimizes the weighted sum of the distances between all pairs of facilities:

$$\min_{\pi \in \Pi_n} \sum_{1 \leq i < j \leq n} c_{\pi(i)\pi(j)} d_{ij}^\pi \quad (4)$$

where d_{ij}^π , the distance between facility i and j in layout π , is defined as:

$$d_{ij}^\pi = \frac{l_{\pi(i)}}{2} + \sum_{i < k < j} l_{\pi(k)} + \frac{l_{\pi(j)}}{2} \quad (5)$$

Considering Equation (2) with respect to the SRFLP τ_{ij} describes the desirability to assign the facility j to location i and η_{ij} is a problem-specific heuristic proposed by [17] as:

$$\eta_{ij} = \frac{1}{1 + \sum_{h=1}^{j-1} \left((c_{\pi(h)i} + c_{i\pi(h)}) d_{hj}^\pi \right)} \quad (6)$$

This heuristic prefers facilities that cause the least increase in cost when being added to the partial solution constructed so far. The computational effort for the heuristic is high compared to the TSP heuristic because it has to be computed in every solution construction step. Contrary to the TSP where the aim is to find a cyclic permutation (every city could be the first), the SRFLP requires a linear permutation. Thus, the first facility must also be learned by the algorithm. This problem is addressed by adding a dummy facility as the zeroth facility with $l_0 = 0$ and $c_{i0} = c_{0i} = 0$ for $i = 0, \dots, n$ which ensures that the solution quality is not influenced by this dummy facility.

The modification of the ACO guidance process that we propose in this work makes use of so called common intervals, for which we adapted the following definition from [9]. Let $N = \{1, 2, \dots, n\}$ be a set of n elements and π a permutation of N . For $x, y \in [1, n]$ with $x \leq y$ let $[x, y]$ denote the set $\{x, x+1, x+2, \dots, y\}$. An interval of π can then be defined as $\pi([x, y]) = \{\pi(i) \mid i \in [x, y]\}$, i.e. as a subset of the element set N that appears consecutively in π . The length of an interval is defined as the number of contained elements, i.e., $\pi([x, y])$ has the length $y - x + 1$. Given a family of k permutations $\Pi = (\pi_1, \dots, \pi_k)$ a *common interval* C is a subset of the element set N that is an interval in all permutations, i.e. $\exists x_1, \dots, x_k, y_1, \dots, y_k : C = \pi_1[x_1, y_1] = \dots = \pi_k[x_k, y_k]$. In the following, the trivial intervals $\{1\}, \{2\}, \dots, \{n\}$ and $\{1, 2, \dots, n\}$ are not considered.

It has to be mentioned that the aim of this paper is to study a novel solution construction method and not to improve known best solutions. Therefore, our algorithm does not use local

search or precomputed initial solutions, i.e., solutions that were built with the help of a heuristic ahead of the optimization process.

III. ACO WITH COMMON INTERVALS

In this section we describe the modifications of the ACO algorithm that are necessary for the proposed CIACO algorithm.

CIACO uses a solution construction process that differs from standard ACO. A new kind of guidance is used that allows the ants to communicate indirectly through the solutions they have found. A pseudocode of CIACO can be found in Algorithm 2 and the algorithm parameters are listed in Figure 1. After every iteration the set of common intervals of the ψ best unique solutions are computed by using the method from [10]. After that, all intervals with length $< \theta$ are removed where θ is a parameter of the algorithm. The resulting set of common intervals of length $\geq \theta$ is used to guide the ants. Since the shortest common intervals have a length of two the case of $\theta = 2$ means that no common intervals are removed. Note that no solutions are available in the first iteration of CIACO to compute common intervals. Thus, the solution construction in the first iteration is performed as in the standard ACO algorithm. This may also happen in later iterations if less than ψ , unique solutions are available. This happens, for example, when the algorithm has converged. For the TSP the solution construction starts by selecting a random initial city. In case of the SRFLP where a linear order has to be constructed the solution construction process starts with the zeroth facility placed on the zeroth location.

If a chosen item (city for TSP, facility for SRFLP) lies within a common interval than the ant has to chose all items in this common interval first before moving on to other items outside the common interval. However, the order of visiting the items within an interval is not prescribed but is determined by using the pseudo-random-proportional rule. Hence, the ant's only constraint is to visit the items in the common interval, whereas the order of these items is not predefined. From another point of view the common intervals contain information about *locality* but not necessarily *quality*. Thus, it could be beneficial to visit this specific set of items sequentially but different orders of the items could result in different solution qualities. If there is more than one common interval that contains *currentNode* (see Algorithm 2) the shortest among those common intervals is selected for guidance. Since the larger intervals are considered afterwards this guarantees that all common intervals are regarded.

Sometimes the constraint, to chose all items within a common interval before choosing any other items, can be too rigorous and may negatively effect the ability of the ants to explore the search space. To slightly relax the common interval constraint an additional probability $p^{guide} \in [0, 1]$ was introduced that determines the probability that the ants are allowed to violate the restriction for the following construction step. A value of p^{guide} that is close to 0 means that violations are more likely. The effect is that the guidance by the common

- $T = \emptyset$: the tour
- $A = \{1, 2, 3, \dots, n\}$: available nodes
- ψ : number of the ants' solutions used to compute the common intervals
- θ : threshold for removing shorter common intervals
- C : all common intervals (computed and filtered with ψ and θ)
- C' : set of common intervals that contain the *currentNode*
- C : the common interval that will be used for the guidance of an ant
- p^{guide} : probability to break out, from the common interval guidance, for one construction step

Figure 1. Overview of the parameters for the CIACO algorithm

intervals is relaxed. Values of p^{guide} close to 1 result in a strong guidance.

IV. EXPERIMENTAL SETUP

In this section the test instances and parameter values that were used for the simulations are described.

A. Test Instances

Five TSP test instances from the TSPLIB (accessible under <http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/>) were used. The chosen problem instances are evenly distributed within a ranking with respect to the number of cities, from those instances with euclidean distances in 2D up to a size of 400 cities. Since the largest of these instances (*rd400*) required an unreasonable long run time we replaced it with *bier127* and *ch130*. This resulted in the following instances: *eil51*, *kroB100*, *bier127*, *ch130*, *pr144*, and *ts225*. For the SRFLP we used the five instances each with 30 facilities that have been published in [18].

B. Parameter Values

The following parameter values were used in the simulations: $\alpha = 1.0$, $\beta \in \{0.0, 1.0, 2.0\}$, $\rho = 0.05$, $\psi \in \{2, 3, 4\}$, $\theta \in \{2, 3, 4\}$, $q_0 \in \{0.25, 0.5, 0.75\}$, $p^{guide} \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$, 20 ants per iteration. Each pheromone value has an initial value of 0.05. Hence, overall 405 parameter combinations were tested and for each combination the results are averaged over 20 runs. For each run the maximum number iterations was 1000.

V. EXPERIMENTAL RESULTS AND DISCUSSION

In this section the results of CIACO and ACO are compared and the properties of the CIACO algorithm are analyzed.

Evaluation of the Results

A general problem of optimization algorithms is the right choice of parameter values. Important parameters for the CIACO are ψ , θ , and p^{guide} . Therefore, the influence of these parameters on the solution quality has been investigated. The values for parameters α, β, ρ, q_0 , the number of ants, and

Algorithm 2 CIACO Solution Construction

```

1: function MAIN
2:   initialize()
3:   while not termination criterion is met do
4:      $C \leftarrow \text{createCommonIntervals}(\psi, \theta)$ 
5:     if  $|C| = 0$  then
6:       constructSolutionsACO()
7:     else
8:       constructSolutionsCIACO()
9:     end if
10:    updatePheromones()
11:  end while
12: end function
13:
14: function CREATECOMMONINTERVALS( $\psi, \theta$ )
15:   // get common intervals of the  $\psi$  best unique solutions
16:   computeCommonIntervals( $\psi$ )
17:   // remove all common intervals with a length  $< \theta$ 
18:   filterCommonIntervals( $\theta$ )
19: end function
20:
21: function CONSTRUCTSOLUTIONSCIACO
22:   for all ants do
23:     repeat
24:       if  $|T| = 0$  then
25:         // first node
26:          $currentNode = \text{getRandomNode}(A)$ 
27:       else
28:         // use pseudo-random-proportional rule
29:          $currentNode = \text{getNextNode}(A)$ 
30:       end if
31:        $T \leftarrow T \cup \{currentNode\}$ 
32:        $A \leftarrow A \setminus \{currentNode\}$ 
33:        $C \leftarrow \text{findShortestCommonInterval}(currentNode, C)$ 
34:        $p \leftarrow \text{getRandomUniform}(0, 1)$ 
35:       if  $(C \neq \emptyset) \wedge (p \leq p^{guide})$  then
36:          $C \leftarrow C \cap A$ 
37:         visitAllNodes( $C$ )
38:          $C \leftarrow C \setminus C$ 
39:       end if
40:     until isComplete( $T$ )
41:   end for
42: end function
43:
44: function VISITALLNODES( $C$ )
45:   while  $C \neq \emptyset$  do
46:     // use pseudo-random-proportional rule
47:      $currentNode = \text{getNextNode}(C)$ 
48:      $A \leftarrow A \setminus \{currentNode\}$ 
49:      $T \leftarrow T \cup \{currentNode\}$ 
50:      $C \leftarrow C \setminus \{currentNode\}$ 
51:   end while
52: end function

```

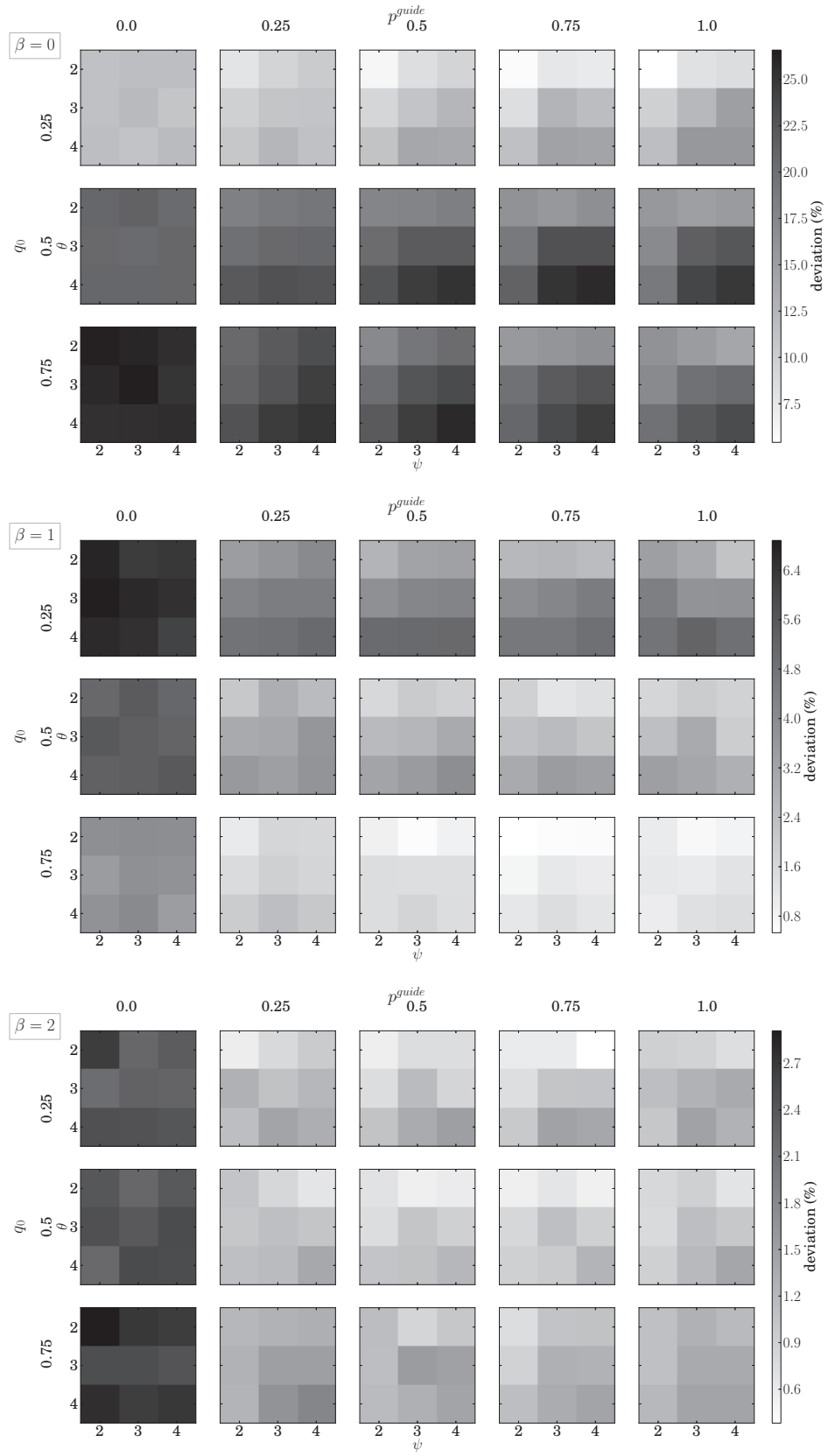


Figure 2. Average relative performance (in percent) of CIACO for the TSP for different parameter combinations with respect to best parameter value combination with $\beta = 0.0$ (top) , $\beta = 1.0$ (middle), $\beta = 2.0$ (bottom); a brighter color is better

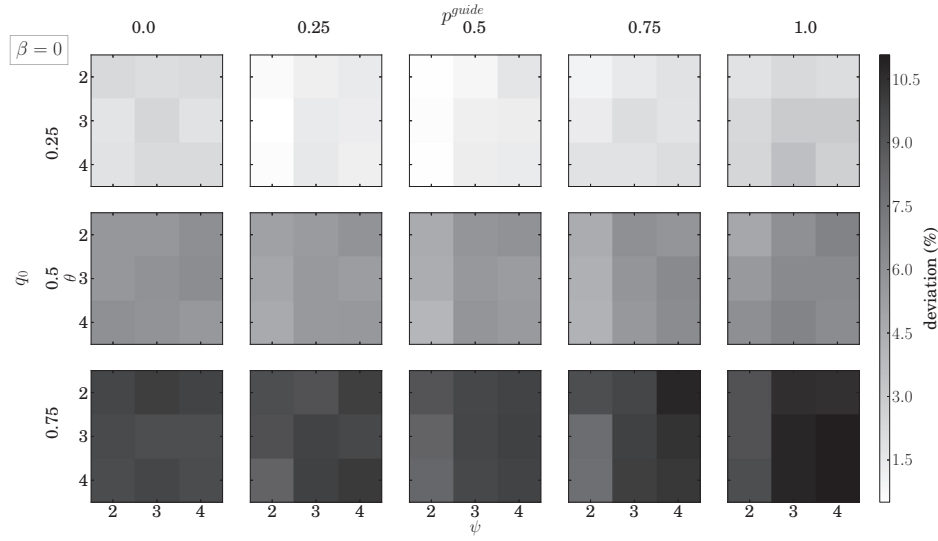


Figure 3. Average relative performance (in percent) of CIACO for the SRFLP for different parameter combinations with respect to best parameter value combination with $\beta = 0.0$; a brighter color is better

```

53: function FINDSHORTESTCOMMONINTER-
    VAL(currentNode, C)
54:    $\mathcal{C}' \leftarrow \{C \mid C \in \mathcal{C} \wedge (C \cap \{currentNode\}) \neq \emptyset\}$ 
55:   if  $\mathcal{C}' \neq \emptyset$  then
56:     return  $\arg \min_{C \in \mathcal{C}'} |C|$ 
57:   else
58:     return  $\emptyset$ 
59:   end if
60: end function

```

number of iterations that have been used in the tests are the same for ACO and CIACO to provide a fair comparison.

For the evaluation of the results two measures were used. First for the comparison of the ACO and the CIACO algorithm we defined the quality as follows. For a certain problem instance and algorithm using a given combination of parameter values (β , q_0 and additionally ψ , θ , and p^{guide} for CIACO) the quality is the average solution quality over 20 runs. In order to evaluate the influence of a certain parameter on the optimization behavior of CIACO the relative performance of different combinations of parameter values is measured. For each fixed value of β and every combination of parameter values for ψ , θ , q_0 , and p^{guide} the relative decrease (in percent) in quality is measured with respect to the smallest quality value obtained for the respective value of β over all considered parameter combinations of ψ , θ , q_0 , and p^{guide} . This is done for every test instance. Then for each parameter combination the average of all the relative deviations (in percent) over the test instances is computed. We call the obtained value the average relative deviation.

Table I
THE BEST SOLUTIONS FOR THE ACO AND CIACO ALGORITHMS PER TSP INSTANCE. SUPERIOR SOLUTIONS ARE WRITTEN IN BOLDFACE.

		β	q_0	ψ	θ	p^{guide}	quality
eil51	ACO	2.0	0.25	-	-	-	443.77
	CIACO	2.0	0.25	2	4	0.75	434.66
kroB100	ACO	2.0	0.25	-	-	-	23516.98
	CIACO	2.0	0.25	3	2	0.75	22847.40
bier127	ACO	2.0	0.25	-	-	-	124074.05
	CIACO	2.0	0.25	3	2	0.5	122661.4
ch130	ACO	2.0	0.5	-	-	-	6523.21
	CIACO	2.0	0.25	4	2	0.75	6422.45
pr144	ACO	2.0	0.75	-	-	-	60028.09
	CIACO	2.0	0.75	2	2	0.25	59454.52
ts225	ACO	2.0	0.5	-	-	-	132976.6
	CIACO	2.0	0.5	2	2	0.75	130109.25

Table II
THE BEST SOLUTIONS FOR THE ACO AND CIACO ALGORITHMS PER SRFLP INSTANCE. SUPERIOR SOLUTIONS ARE WRITTEN IN BOLDFACE.

		β	q_0	ψ	θ	p^{guide}	quality
Nugent-1	ACO	0.0	0.25	-	-	-	9116.95
	CIACO	0.0	0.25	4	3	0.25	8955.8
Nugent-2	ACO	0.0	0.25	-	-	-	24026.25
	CIACO	0.0	0.25	2	2	0.25	23616.95
Nugent-3	ACO	0.0	0.25	-	-	-	50894
	CIACO	0.0	0.25	2	2	0.75	49761.5
Nugent-4	ACO	0.0	0.25	-	-	-	64419.75
	CIACO	0.0	0.25	2	3	0.5	62874.6
Nugent-5	ACO	0.0	0.25	-	-	-	128918.55
	CIACO	0.0	0.25	2	4	0.5	125146.2

Tables I and II show the best solution qualities and their corresponding parameter values obtained by ACO and CIACO for the TSP and the SRFLP, respectively. For both problems and all test instances the CIACO algorithm is superior to the ACO algorithm. This clearly shows that the common interval guidance was helpful to find good solutions. For the CIACO the best qualities have been found for $p^{guide} \geq 0.25$. This also shows that a significant consideration of the common intervals is useful. For all test instances of the TSP both algorithms find the best solutions with $\beta = 2.0$, i.e., when the influence of the heuristic is large. It has been shown in other studies on ACO that heuristic guidance is useful for the TSP. Contrary to this, the best solutions for the SRFLP were found for all test instances without using the heuristic, i.e., with $\beta = 0.0$. Thus the heuristic did not work well for the analyzed problem instances. Note, that in [17] where the heuristic was proposed, the influence of the heuristic on the algorithm was actually chosen to be small (the heuristic was weighted only with 0.2; $\alpha = 0.8$ [17] p. 594).

Analysis of CIACO for TSP and SRFLP

Figure 2 shows the aggregated results for the TSP test instances and $\beta = 0.0$, $\beta = 1.0$, and $\beta = 2.0$, respectively. Note that the absolute values and the range of the average relative deviations decreases with increasing values of β . While for $\beta = 2.0$ this range is 21.17% it is only 6.35% and 2.52% for $\beta = 1.0$ and $\beta = 2.0$, respectively. Hence, with a strong heuristic influence the relative performance differences of the different parameter combinations from each other decrease. A general trend that can be observed for all values of β is that the results obtained with the common interval guidance, i.e., $p^{guide} > 0$, are superior to the results obtained without guidance, i.e., for $p^{guide} = 0.0$. Furthermore, it can be observed that for most combinations of β , p^{guide} , and q_0 smaller values of θ and ψ tend to yield better results. This suggests that the use of the smaller common intervals is helpful, but if they are computed for too many of the good solutions (i.e., for large ψ values) then the solution quality decreases.

When no heuristic is used, i.e., for $\beta = 0.0$, then an emphasis on exploration, i.e., $q_0 = 0.25$, yields the best results. Here the smallest average relative deviations can be found in the first row and the first column of the corresponding submatrices, i.e., for $\theta = 2$ and $\psi = 2$. The overall smallest average relative deviation (5.42%) was achieved using the parameters $q_0 = 0.25$, $p^{guide} = 1.0$, $\psi = 2$, $\theta = 2$. This means that the algorithm preferred exploration to exploitation, was strongly guided by the common intervals, used only two solutions to construct the common intervals, and used all available intervals (i.e., no common intervals were discarded due to θ).

In contrast, for $\beta = 1.0$ the best solutions are found with q_0 set to 0.75, i.e., exploration turned out to favor good solutions, and the solution quality decreases with decreasing q_0 values. The overall smallest average relative deviation (0.53%) was

obtained with the parameters $q_0 = 0.75$, $p^{guide} = 1.0$, $\psi = 2$, $\theta = 2$ meaning that the algorithm was strongly guided by the common intervals, used two solutions for the common interval construction, and did not discard any of the shorter common intervals.

Yet another behavior was observed for $\beta = 2.0$. Here the variation of an explorative vs. exploitive scheme did not yield any obvious differences. Rather the differences in the average relative deviations were mainly due to the variation of the influence of the common intervals, i.e., different values of p^{guide} . Observe, that the range of the average relative deviation is smaller than for $\beta = 0.0, 1.0$. The overall smallest average relative deviation (0.38%) was achieved with parameter values $q_0 = 0.25$ (more exploration), $p^{guide} = 0.75$ (strong common interval guidance), $\psi = 4$ (using four solutions to construct the common intervals) and $\theta = 2$ (using common intervals of all lengths).

Figure 3 shows the aggregated results for SRFLP and $\beta = 0$. The figures for $\beta \in \{1.0, 2.0\}$ were omitted because the differences to the case $\beta = 0$ are marginal. Hence, the influence of the heuristic seems to be negligible (as discussed above). As can be observed from the row with $q_0 = 0.25$ the average relative deviations are larger for $p^{guide} = 0.0$, i.e., no guidance, and become smaller for $p^{guide} \in \{0.25, 0.5\}$. They become larger again for $p^{guide} \in \{0.75, 1.0\}$, i.e., full guidance. This confirms the positive influence of the common interval guidance and also the intuition that too much guidance, i.e., $p^{guide} \in \{0.75, 1.0\}$, can affect the solution quality negatively. Finally, the overall smallest average relative deviation (0.50%) can be found for $q_0 = p^{guide} = 0.25$, $\psi = 2$ and $\theta = 3$. This implies that the CIACO algorithm preferred exploration to exploitation, was weakly guided by the common intervals, used two solutions to construct the common intervals and discarded common intervals of length two.

VI. CONCLUSION

We presented a new solution construction scheme for ACO which can be applied to permutation problems. This scheme made use of the common intervals constructed from the best solutions in each iteration. An empirical study was conducted a) to compare the optimization behavior of ACO and the proposed CIACO algorithm and b) to investigate the influence of the additional parameters ψ , θ , and p^{guide} on the CIACO algorithm. We showed that the solutions found by the CIACO algorithm were superior to the solutions found by the ACO algorithm for all test instances of both test problems (the TSP and the SRFLP). We furthermore demonstrated that the use of common interval guidance generally improves the solution quality. The results also show that it is advantageous to use a small number of good solutions for the construction of the common intervals and that using small common intervals for the guidance is advantageous.

ACKNOWLEDGMENTS

This work has been supported by the European Social Fund (ESF) and the Free State of Saxony within Nach-

REFERENCES

- [1] P.-P. Grassé, “La reconstruction du nid et les coordinations interindividuelles chez *bellicositermes natalensis* et *cubitermes* sp. la théorie de la stigmergie: Essai d’interprétation du comportement des termites constructeurs,” *Insectes Sociaux*, vol. 6, pp. 41–80, 1959.
- [2] M. Dorigo and T. Stützle, *Ant Colony Optimization*. Scituate, MA, USA: Bradford Company, 2004.
- [3] J.-L. Deneubourg, S. Aron, S. Goss, and J. Pasteels, “The self-organizing exploratory pattern of the argentine ant,” *Journal of Insect Behavior*, vol. 3, no. 2, pp. 159–168, 1990.
- [4] D. Goldberg, *Genetic algorithms in search, optimization, and machine learning*, ser. Artificial Intelligence. Addison-Wesley, 1989.
- [5] R. Brady, “Optimization strategies gleaned from biological evolution,” *Nature*, vol. 317, no. 6040, pp. 804–806, 1985.
- [6] H. Mühlenbein, M. Gorges-Schleuter, and O. Krämer, “Evolution algorithms in combinatorial optimization,” *Parallel Computing*, vol. 7, no. 1, pp. 65–85, 1988.
- [7] S. Kobayashi, I. Ono, and M. Yamamura, “An efficient genetic algorithm for job shop scheduling problems,” in *Proceedings of the 6th International Conference on Genetic Algorithms*, 1995, pp. 506–511.
- [8] T. Uno and M. Yagiura, “Fast algorithms to enumerate all common intervals of two permutations,” *Algorithmica*, vol. 26, no. 2, pp. 290–309, 2000.
- [9] S. Heber and J. Stoye, “Finding all common intervals of k permutations,” in *Proceedings of the 12th Annual Symposium on Combinatorial Pattern Matching*, ser. CPM ’01, 2001, pp. 207–218.
- [10] A. Bergeron, C. Chauve, F. de Montgolfier, and M. Raffinot, “Computing common intervals of k permutations, with applications to modular decomposition of graphs,” *SIAM Journal on Discrete Mathematics*, vol. 22, no. 3, pp. 1022–1039, 2008.
- [11] G. Fertin, A. Labarre, I. Rusu, E. Tannier, and S. Vialette, *Combinatorics of Genome Rearrangements*. MIT Press, 2009.
- [12] M. Bernt, A. Braband, M. Middendorf, B. Misof, O. Rota-Stabelli, and P. F. Stadler, “Bioinformatics methods for the comparative analysis of metazoan mitochondrial genome sequences,” *Molecular Phylogenetics and Evolution*, 2012, accepted.
- [13] T. Stützle and M. Dorigo, “ACO algorithms for the traveling salesman problem,” in *Evolutionary Algorithms in Engineering and Computer Science: Recent Advances in Genetic Algorithms, Evolution Strategies, Evolutionary Programming, Genetic Programming and Industrial Applications*, K. Miettinen, Ed. John Wiley & Sons, 1999.
- [14] M. Dorigo and L. Gambardella, “Ant colony system: a cooperative learning approach to the traveling salesman problem,” *Evolutionary Computation, IEEE Transactions on*, vol. 1, no. 1, pp. 53–66, 1997.
- [15] D. M. Simmons, “One-dimensional space allocation: An ordering algorithm,” *Operations Research*, vol. 17, no. 5, pp. 812–826, 1969.
- [16] R. Kothari and D. Ghosh, “The single row facility layout problem: state of the art,” *OPSEARCH*, pp. 1–21, 2012.
- [17] M. Solimanpur, P. Vrat, and R. Shankar, “An ant algorithm for the single row layout problem in flexible manufacturing systems,” *Comput. Oper. Res.*, vol. 32, no. 3, pp. 583–598, 2005.
- [18] C. E. Nugent, T. E. Vollmann, and J. Ruml, “An experimental comparison of techniques for the assignment of facilities to locations,” *Operations Research*, vol. 16, no. 1, pp. 150–173, 1968.